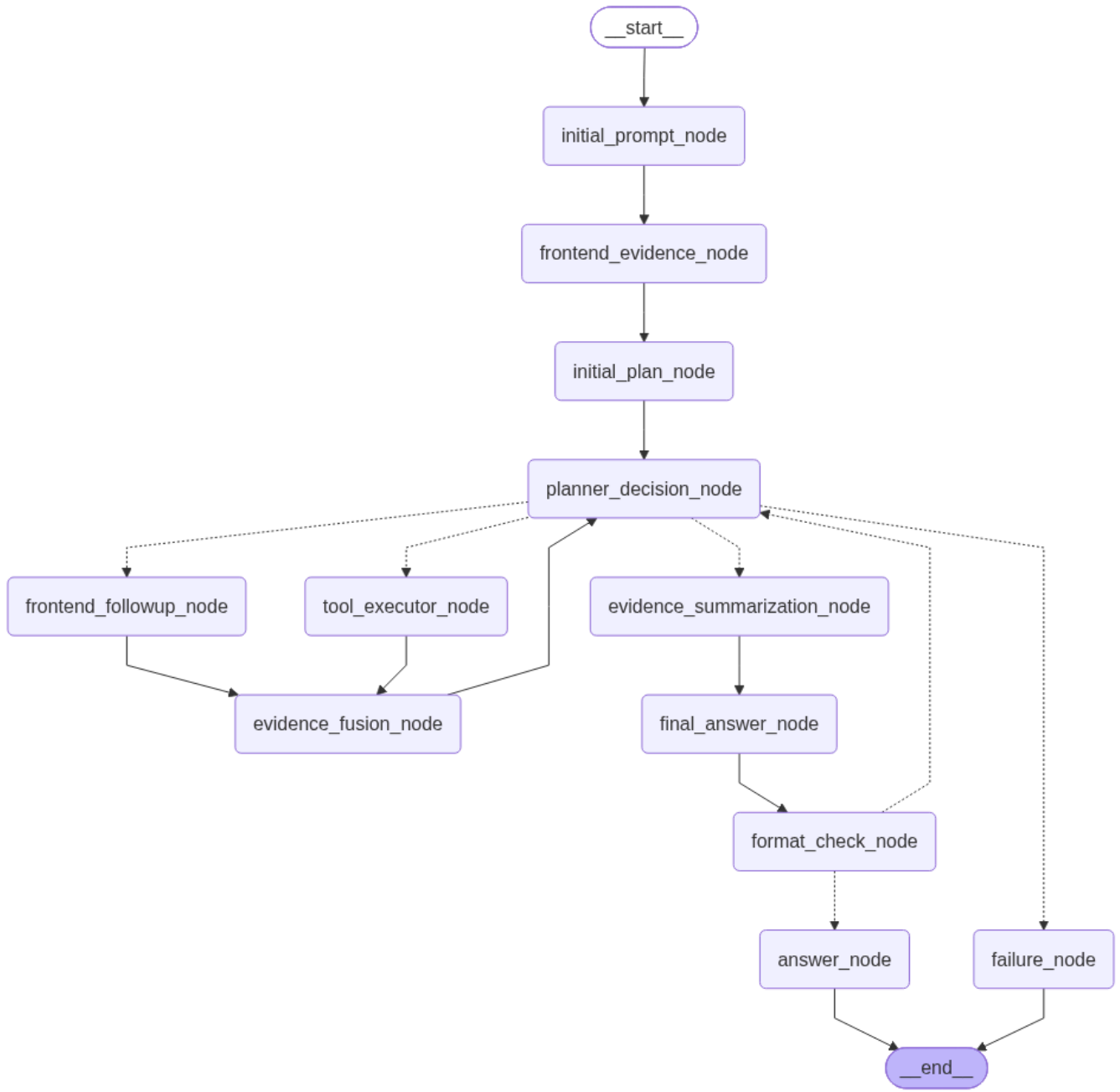


- 一、Audio Mind 的设计
- 二、实验设计和结果
 - 2.1 MMAR
 - 2.1.1 setup
 - 2.1.2 results
 - 2.1.3 为什么之前的优秀系统(比如AudioGenie-Reasoner)跑分还不如单独的frontend?
 - 2.1.4 工具调用的行为分析
 - 2.1.5 MMAR-Rubrics的跑分
 - 2.1.6 在具体子分类的跑分
 - 2.1.7 每个工具的调用频率分析（重尾）
 - 2.1.8 re-listening次数分析
 - 2.1.9 综合结论
 - 2.2 MSU Bench
 - 2.2.1 setup
 - 2.2.2 results
 - 2.2.3 结论
- 三、工程阅读和复现情况
- 四、总结

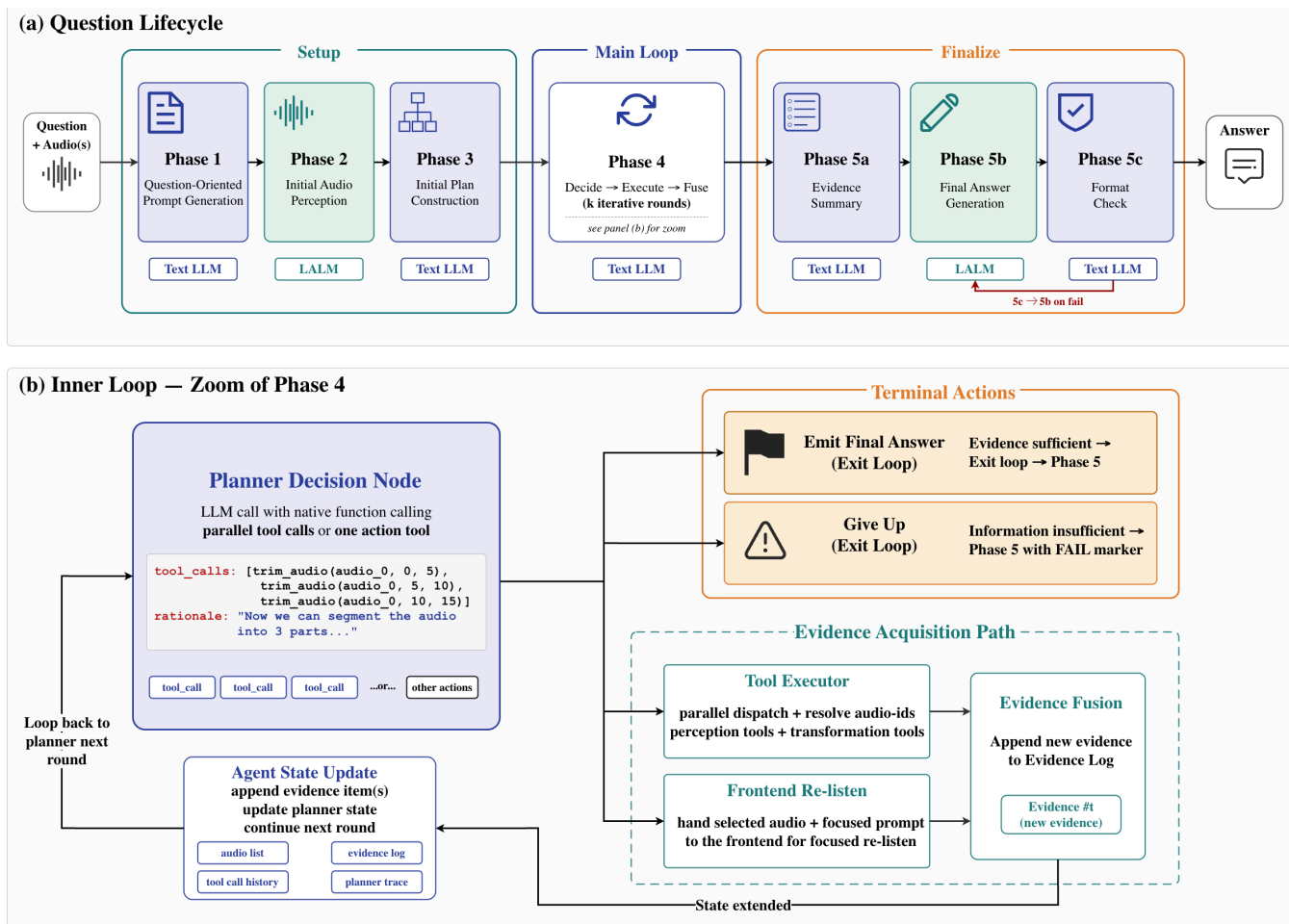
Audio Mind 是一个 audio understanding 领域的可审计、可插拔的agentic framework.

一、Audio Mind 的设计

如下图所示，该图是 langgraph 自己自带的可视化工具产生的audio mind的数据流节点图：



和论文里的图一致：



总的来说，Audio Mind是这样设计：

- 1.先把音频和问题给到text llm，并设计精良的提示词工程，让text llm产生比较精良的QoP（Question-Oriented Prompt），本质是对原问题进行精良加工。
- 2.把加工后的QoP给到LALM，让frontend先回答初始的音频感知信息。
- 3.再把初始问题，初始frontend感知信息，以及设计好的精良提示词喂给text llm，让text llm产出一份初始的决策计划，包含可能依赖的工具能力、决策的依赖顺序参考等。
- 4.拿到初始计划后，text llm再根据设计精良的提示词工程，以及toolspec等信息，进行回答问题/承认失败/前端重感知/选择并执行工具的循环。
- 5.如果选择前端重感知和执行工具，则都会产生新的evidence,把新的evidence融合到共享状态，供下次循环做决策使用。注意是evidence而不是最终结果，模型被告知evidence以及评估这条evidence的置信度水平，来综合考虑是否有把握回答问题。
- 6.如果选择回答问题，那么首先text llm会使用设计好的精良的提示词工程，以及evidence列表信息先总结，提取出一版总结摘要，过滤掉无用的思考决策过程、无用信息等信息。然后再把初始问题，提取出的总结摘要配合精良的提示词工程一同喂给frontend，产生final answer，最后text llm进行格式检查，无误则本task完成，有误则把错误信息当成evidence append到共享状态，重新进入decision loop。
- 7.如果选择承认失败则终止。一般是满足了max_steps到达上限等终止条件。

二、实验设计和结果

2.1 MMAR

2.1.1 setup

- Frontend : `qwen-3.5-omni`
- Text LLM : `qwen3.5`

2.1.2 results

Method	Sound	Music	Speech	So-Mu	So-Sp	Mu-Sp	So-Mu-Sp	Avg.
End-to-end models								
GPT-4o Audio (Hurst et al., 2024)	53.94	50.97	70.41	63.64	72.48	62.20	75.00	63.50
Gemini 2.0 Flash (Google, 2025)	61.21	50.97	72.11	81.82	72.48	65.85	70.83	65.60
Gemini 2.5 Pro (Comanici et al., 2025)	67.30	56.80	82.00	100.00	84.90	80.50	66.70	74.70
Gemini 3 Pro (Google DeepMind, 2026)	72.73	61.65	84.35	72.73	81.19	78.05	83.33	76.40
Qwen3.5-Omni (Qwen Team, 2026)	—	—	—	—	—	—	—	80.00
Qwen3.5-Omni [†] (Qwen Team, 2026)	75.76	61.17	85.37	90.91	87.61	79.27	87.50	78.90
Audio agents								
AudioRouter (Chen et al., 2026)	61.40	66.10	77.20	57.20	65.10	68.30	55.50	64.40
AuTAgent (Tong et al., 2026)	51.52	45.15	73.81	—	—	—	—	63.00
AudioToolAgent (Wijngaard et al., 2025)	73.33	63.59	82.99	63.64	83.03	81.71	79.17	77.00
AudioGenie-Reasoner (Rong et al., 2025)	49.68	43.26	69.23	45.45	64.53	65.33	59.09	58.85
AudioGenie-Reasoner [†] (Matched Backbone) (Rong et al., 2025)	70.91	54.85	79.59	54.55	72.48	76.83	58.33	70.50
AUDIO-MIND [†]	78.18	63.11	87.41	100.00	87.61	79.27	87.50	80.40

Table 2: MMAR accuracy by modality. So, Mu, and Sp denote sound, music, and speech. Rows marked with [†] are evaluated by us on the same 1,000 MMAR examples; other values are taken from the corresponding benchmark or system reports.

2.1.3 为什么之前的优秀系统(比如AudioGenie-Reasoner)跑分还不如单独的frontend?

AudioGenie-Reasoner 在 2025 年是对的，那时候基座只有 7B，工具虽然有偏见，但比"听不懂"强。它的论文价值在于"如何把弱基座的能力榨干"。但到了 2026 年，基座变成了 Qwen3.5-Omni 这种级别的模型，同样的流水线设计反而成了枷锁——它把强基座当成弱基座用，硬生生把 78 分的水平压到了 70 分。AUDIO-MIND 的核心贡献就是指出：强基座时代，agentic 设计要从"给弱基座补短板"转向"别挡着强基座发挥"。

2.1.4 工具调用的行为分析

目的：验证tool use是否真的按照条件证据收集的角色作用。MMAR一共1k个task,其中一个由于护栏走入了承认失败分支，所以取999个样本的轨迹分析，每个问题平均1.68次tool call，31.2%的问题没有调用任何工具，所以这个框架确实是条件性地证据收集。当前端证据看起来不够时，工具调用深度增加；当前端证据看起来够时，工具调用保持低水平。这正是 AUDIO-MIND 设计的理想行为：

- 简单问题 → 前端自己搞定，不折腾工具
- 中难问题 → 精准补几个工具，显著提升准确率
- 极难问题 → 工具也救不了，但至少系统尝试过（虽然这里可能过度调用了）

Rounds	Count	Share
1	311	31.1%
2	255	25.5%
3	164	16.4%
4–5	203	20.3%
6–10	59	5.9%
>10	7	0.7%

Table 6: Distribution of planner rounds per MMAR question.

Tool calls	N	AUDIO-MIND	Qwen3.5-Omni	Δ
0	311	89.7	90.4	-0.6
1	255	85.5	86.3	-0.8
2	164	73.8	72.6	+1.2
3	142	72.5	66.2	+6.3
4–5	96	69.8	60.4	+9.4
6–10	25	56.0	52.0	+4.0
>10	6	16.7	50.0	-33.3

Table 4: MMAR accuracy comparison between AUDIO-MIND and Qwen3.5-Omni, grouped by the number of tool calls made by AUDIO-MIND. Accuracy and Δ are reported in percentage points.

这张表证明了 AUDIO-MIND 的"条件性"不是纸上谈兵：规划器真能感知到前端哪里不懂，并据此决定调多少工具。工具调用越多的题，本质上越难；工具在"该用的时候"确实有用，在这些难题上，AUDIO-MIND 比强基座直接推理的优势最大（高出近10分）。只有极少数极端难题（0.6%）会出现过度调用，这说明规划器的"止损机制"还有改进空间，当工具调用超过一定次数还没补上缺口时，应该直接放弃或输出"无法确定"，而不是继续调。可能的解释是多个工具结果证据让前端分心了。

2.1.5 MMAR-Rubrics的跑分

AUDIO-MIND : 66.5%
only frontend Qwen3.5-Omni : 59.6%

2.1.6 在具体子分类的跑分

Layer	Sub-category	N	Qwen	AudioGenie	AUDIO-MIND	Δ vs. Qwen	Δ vs. AudioGenie
Cultural	Aesthetic Evaluation	8	75.00	50.00	62.50	-12.50	+12.50
Cultural	Culture of Speaker	52	78.85	76.92	82.69	+3.85	+5.77
Cultural	Imagination	10	70.00	60.00	80.00	+10.00	+20.00
Cultural	Professional Knowledge and Reasoning	71	76.06	74.65	76.06	+0.00	+1.41
Perception	Correlation Analysis	50	80.00	74.00	88.00	+8.00	+14.00
Perception	Counting and Statistics	99	64.65	56.57	63.64	-1.01	+7.07
Perception	Environmental Perception and Reasoning	149	87.25	70.47	85.91	-1.34	+15.44
Perception	Music Theory	63	57.14	55.56	73.02	+15.87	+17.46
Perception	Spatial Analysis	15	73.33	73.33	60.00	-13.33	-13.33
Perception	Temporal Analysis	28	64.29	42.86	78.57	+14.29	+35.71
Semantic	Content Analysis	304	85.20	80.26	86.84	+1.64	+6.58
Semantic	Emotion and Intention	60	83.33	65.00	76.67	-6.67	+11.67
Semantic	Speaker Analysis	48	87.50	77.08	87.50	+0.00	+10.42
Signal	Acoustic Quality Analysis	18	77.78	55.56	66.67	-11.11	+11.11
Signal	Anomaly Detection	17	70.59	70.59	88.24	+17.65	+17.65
Signal	Audio Difference Analysis	8	62.50	50.00	37.50	-25.00	-12.50

Table 5: Full MMAR sub-category accuracy comparison. Values are accuracies in percentage points. AudioGenie denotes AudioGenie-Reasoner instantiated with Qwen3.5-Omni and Qwen3.5 for matched-backbone comparison.

tool能提供帮助的特定领域跑分增加多，而主要依靠frontend本身的则少甚至倒退。

2.1.7 每个工具的调用频率分析（重尾）

Tool	Calls	% of calls	Q-share
transcribe_qwenasr_with_timestamps	227	14.9	18.9
inspect_audio_plots	206	13.6	11.1
trim_audio	173	11.4	6.9
transcribe_fireredasr_with_timestamps	113	7.4	11.3
analyze_pitch	107	7.0	5.4
analyze_onsets	85	5.6	7.5
transcribe_whisperx_with_diarization	64	4.2	6.2
extract_rms_energy	58	3.8	3.7
analyze_spectral_features	55	3.6	3.8
transcribe_qwenasr	46	3.0	3.7

Table 7: Top-10 most invoked tools on MMAR. Q-share is the percentage of questions invoking the tool at least once.

虽然是重尾分布，但是每个工具都至少被调用过一次，证明了工具列表不是冗余的能力堆积。

2.1.8 re-listening次数分析

re-listening 50/999, for 64 re-listens in total.这和re-listening的角色是一致的, 不是通用的fallback,而是真的一种条件性的获取证据的方式。

2.1.9 综合结论

- Audio Mind是新SOTA, 在MMAR的string_match和MMAR-Rubrics上都是如此
- 强基座时代, agentic 设计要从"给弱基座补短板"转向"别挡着强基座发挥"。
- Audio Mind确实是条件性地证据收集。
- Audio Mind性能的提升依赖任务的特定性: tool能提供帮助的特定领域跑分增加多, 而主要依靠frontend本身的则少甚至倒退。
- 每个工具的调用频率分布是重尾的, 但是每个工具都至少被调用过一次, 证明了工具列表不是冗余的能力堆积。

2.2 MSU Bench

2.2.1 setup

- Frontend: `gemini-2.5-pro`
- Text LLM: `gemini-3.0-flash`

2.2.2 results

Model	Avg.
<i>End-to-end models</i>	
Kimi-Audio (Ding et al., 2025)	0.460
StepAudio2 (Wu et al., 2025)	0.480
MiMoAudio (Zhang et al., 2025)	0.600
Gemini 2.5 Flash [†] (Comanici et al., 2025)	0.682
Qwen3.5-Omni [†] (Qwen Team, 2026)	0.763
Gemini 2.5 Pro (thinking) [†] (Comanici et al., 2025)	0.819
<i>Agentic systems</i>	
AudioGenie-Reasoner [†] (Matched Backbone) (Rong et al., 2025)	0.789
AUDIO-MIND [†]	0.828

Table 3: Average accuracy on MSU-Bench. Rows marked with [†] are evaluated by us; unmarked values are reported by MSU-Bench (Wang et al., 2025). AudioGenie-Reasoner and AUDIO-MIND use the Gemini configuration for MSU-Bench.

2.2.3 结论

- Audio Mind在MSU Bench上是新SOTA

三、工程阅读和复现情况

我已经熟悉了这个工程的数据流流向，对关键逻辑精读了代码。我在windows wsl2 ubuntu 26.04+rtx 5080 laptop上跑通了audio mind。我拉取了我对官方仓库的fork: <https://github.com/ZHYsfl/Audio-Mind>，完全配置好了Audio Mind所需的环境，并且稍微改动以支持blackwell。小量复现两个Bench都用的是阿里云百炼的API (frontend qwen3.5-omni-plus, planner qwen3.5-plus)，与论文的MSU Bench使用Gemini系配置不同，跑分仅作参考。

我在MMAR上小量跑了10个task，跑分如下：

评测方式	跑分
string match	80% (8/10)
MMAR-Rubrics (LLM as judge)	accuracy 80% / mean score 70.0/100

Rubrics 的口径说明 (官方脚本 `evaluation_rubrics.py`，两个数以全部 10 个样本为分母)：

- **accuracy 80%**: 官方规定 judge 只在 string match 通过的样本上打分，没过线的直接判 0 分、不进 judge；`correct` 只表示"进了 judge 并被有效评分"。因此它等于 string match 过线率 (8/10) 。
- **mean score 70.0/100**: judge 按题目自带的 rubric 逐条打分（归一化到 0~1），5 个 rater (NUM_RATERS=5) 各评一次、去掉最高最低取中间 3 个的平均；10 个样本的平均分 ×100 就是 70.0（含 2 个 0 分样本；8 个有效样本的平均分为 87.5）。论文正文的 66.5% 就是这个"含 0 分"的全体平均口径。

补充：第一轮不注入选项时string match只有10%。MMAR官方的string match是字面token匹配而非语义匹配，frontend听懂了也会因为措辞不同（如把"Merry Christmas"意译成中文）得0分。第二轮把4个选项注入prompt并要求"字母+原文"作答后到了80%，与论文口径一致（论文的80.4%同样是注入选项的）。

我在MSU Bench上小量跑了10个task，跑分如下：

指标	跑分
exact-match accuracy	100% (10/10)

10个样本覆盖8种scenario、9种question_type、tier1/2。工程点：（1）全音频统一转16k mono——MSU原始音频长达310s（79MB），base64直传被API拒，16k后9.9MB即可通过；（2）锚点导向题型（no_index/time_index/transcript_index等）按speaker_meta锚点seg裁剪成21~40s窗口再喂，整段型题型（speaker counting/reverse等）保留全音频；（3）裁剪带来的时间轴偏移问题：planner调trim_audio时仍旧用原录音绝对时间（如192.73s），对裁剪后的音频会trim出空文件、工具空转，已在prompt中追加时间偏移说明（in effect:补测到10/10）。第一个样本首轮API 400失败（110s音频base64直传被拒），重试后答对（C），最终10/10。

四、总结

Audio Mind 是一个 audio understanding 领域的可审计、可插拔的agentic framework.我完成了对其工程项目的核心数据流的熟悉、关键逻辑的精读，并配置好环境小量跑了MMAU和MSU Bench的各自10个task的评测。Audio Mind本身通过一系列实验证明了以下等结论：

- Audio Mind是新SOTA，在MMAR的string_match、MMAR-Rubrics和MSU Bench上都是如此
- 当frontend(LALM)变得越来越强的今天，工具已经不能被看成自动提升性能模块，什么时候正确调用工具、如何正确对待工具调用的输出（不一定客观）才有助于性能提升成为新的瓶颈和问题。强基座时代，agentic设计要从"给弱基座补短板"转向"别挡着强基座发挥"。
- Audio Mind确实是条件性地证据收集。
- Audio Mind性能的提升依赖任务的特定性：tool能提供帮助的特定领域跑分增加多，而主要依靠frontend本身的则少甚至倒退。
- 每个工具的调用频率分布是重尾的，但是每个工具都至少被调用过一次，证明了工具列表不是冗余的能力堆积。